Introduction to CP/M Programming

A brief history of the hardware and the software development tools on and for CP/M

What we are covering today

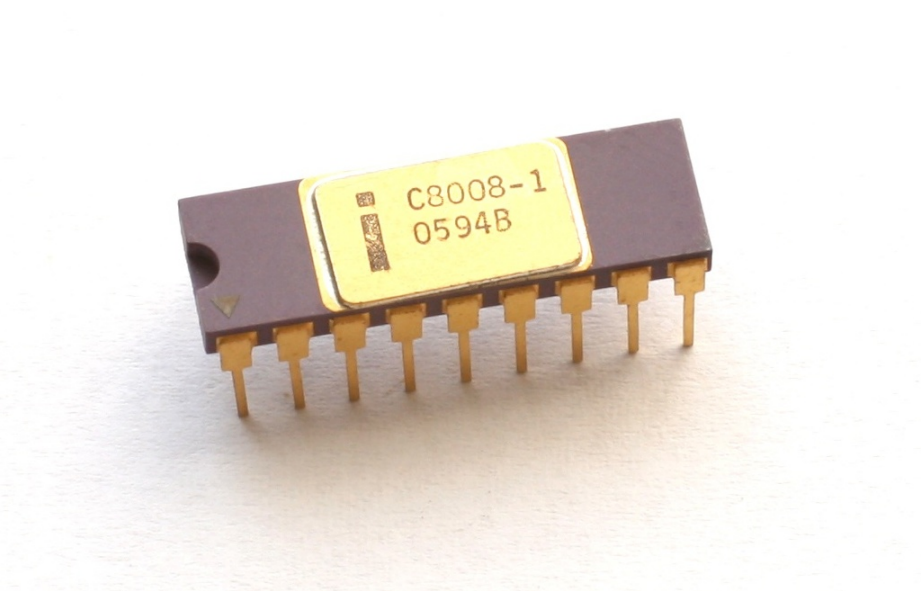
- Introduction to 8-bit Microprocessors
- Brief History of CP/M
- Programming Languages – with examples
- Modern CP/M Tools
- Summary
- Q & A

Introduction to 8-bit Microprocessors

Introduction to 8 Bit Microprocessors

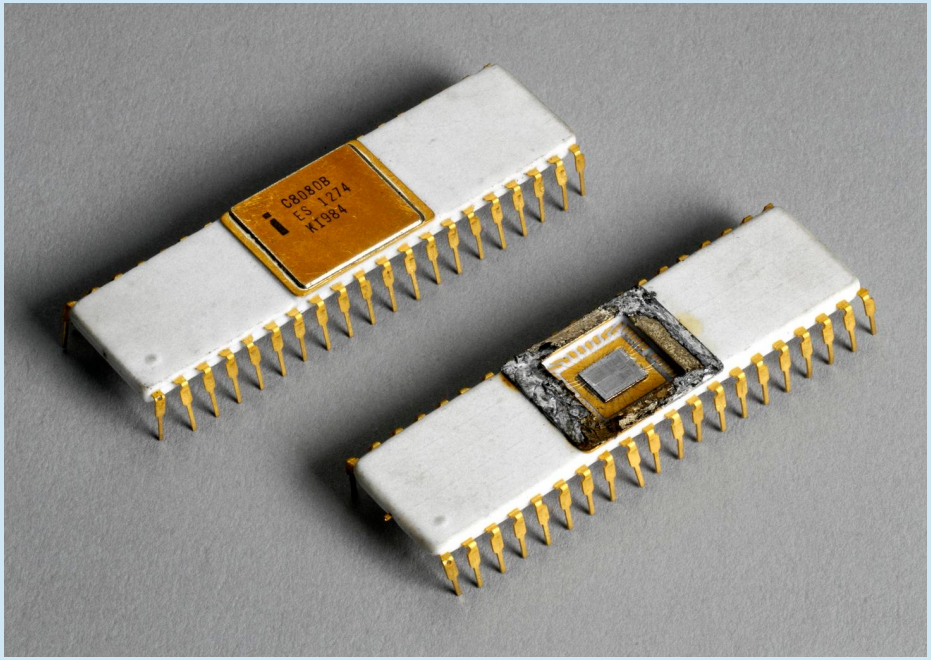
- 8-bit microprocessors were the first widely used microprocessors in the computing industry, marking a major shift from mainframes and minicomputers to smaller, more affordable systems.
  - Intel 8008 – 1972
  - Intel 8080 – 1974
  - Motorola 6800 – 1974
  - MOS Technology 6502 – 1975
  - Zilog Z-80 – 1976

Intel 8008



- The first commercial 8-bit processor was the Intel 8008 in 1972 which was originally intended for the Datapoint 2200 intelligent terminal. Intel's first 8-bit microprocessor, designed by a team including Ted Hoff, Stan Mazor, and Federico Faggin, who had also worked on the 4004.
- The chip, limited by its 18-pin DIP, has a single 8-bit bus working triple duty to transfer 8 data bits, 14 address bits, and two status bits. The small package requires about 30 TTL support chips to interface to memory. For example, the 14-bit address, which can access "16 K × 8 bits of memory", needs to be latched by some of this logic into an external memory address register (MAR). The 8008 can access 8 input ports and 24 output ports.

Intel 8080

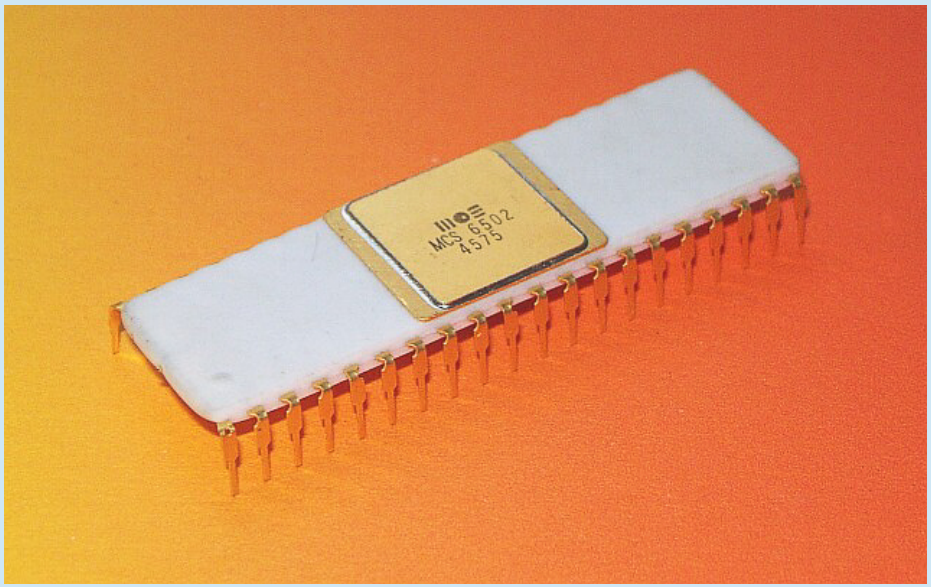


- This was followed by the 8080 in 1974, paving the way for the development of personal computers and the modern computing landscape.
- It was an improved version of the 8008, with a 40-pin package and a clock speed of 2 MHz, it was crucial for early personal computers like the Altair 8800. Most competitors to Intel started off with such character oriented 8-bit microprocessors.
- Modernized variants of these 8-bit machines are still one of the most common types of processor in embedded systems.

Motorola 6800

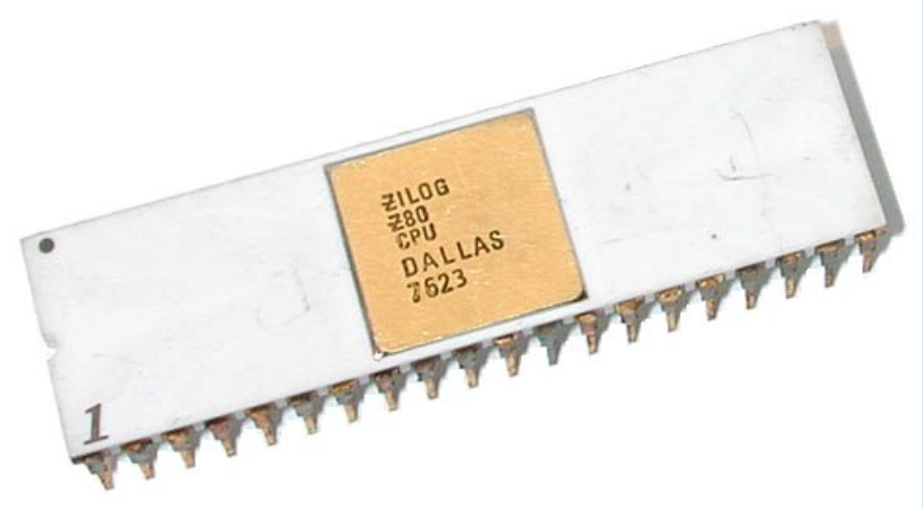
- The Motorola 6800, also introduced in 1974, was another influential design, showcasing innovation and contributing to the democratization of computing.
- Built from scratch, unlike the 8008 and 8080, it took a different approach with fewer registers and a more regular instruction set. It was used in "grown-up" devices like cash machines.
- This was used in the SWTPC 6800 Computer System, an early microcomputer developed by the Southwest Technical Products Corporation and introduced in 1975.

MOS Technology 6502



- The MOS Technology 6502 was released in 1975. When it was introduced, the 6502 was the least expensive microprocessor on the market by a considerable margin.
- It initially sold for less than one-sixth the cost of competing designs from larger companies, such as the 6800 or Intel 8080.
- The 6502 and variants of it, were used in personal computers, such as the Apple I, Apple II, Atari 8-bit computers, BBC Micro, PET, VIC-20, and in home video game consoles such as the Atari 2600 and the Nintendo Entertainment System.

Zilog Z-80



- The Z-80 released in 1976, was developed by Zilog as an enhanced version of the 8080, offering improvements in interrupt handling, instruction set, and memory management.
- The Z-80 was designed to be software-compatible with the Intel 8080, offering a compelling alternative due to its better integration and increased performance.
- Along with the 8080's seven registers and flags register, the Z80 introduced an alternate register set, two 16-bit index registers, and additional instructions, including bit manipulation and block copy/search.

Brief History of CP/M

Brief History of CP/M

- CP/M, originally standing for Control Program/Monitor and later Control Program for Microcomputers, is a mass-market operating system created in 1974 for Intel 8080/8085 based microcomputers by Gary Kildall of Digital Research, Inc.
- CP/M is a disk operating system and its purpose is to organize files on a magnetic storage medium, and to load and run programs stored on a disk.
- Initially confined to single-tasking on 8-bit processors and no more than 64 kilobytes of memory, later versions of CP/M added multi-user variations and were migrated to 16-bit processors.

Brief History of CP/M

- Gary Kildall originally developed CP/M as an operating system to run on an Intel Intellec-8 development system, equipped with a Shugart Associates 8-inch floppy-disk drive interfaced via a custom floppy-disk controller.
- It was written in Kildall's own PL/M (Programming Language for Microcomputers). Various aspects of CP/M were influenced by the TOPS-10 operating system of the DEC system-10 mainframe computer, which Kildall had used as a development environment.
- The CP/M name follows a prevailing naming scheme of the time, as in Kildall's PL/M language, and Prime Computer's PL/P (Programming Language for Prime), both suggesting IBM's PL/I; and IBM's CP/CMS operating system, which Kildall had used when working at the Naval Postgraduate School (NPS).

Brief History of CP/M

- Under Kildall's direction, the development of CP/M 2.0 was mostly carried out by John Pierce in 1978. Kathryn Strutynski, a friend of Kildall from NPS, became the fourth employee of Digital Research Inc. in early 1979. She started by debugging CP/M 2.0, and later became influential as key developer for CP/M 2.2 and CP/M Plus. Other early developers of the CP/M base included Robert "Bob" Silberstein and David "Dave" K. Brown.
- MP/M (Multi-Programming Monitor Control Program) is a multi-user version of the CP/M operating system, created by Digital Research developer Tom Rolander in 1979.It allowed multiple users to connect to a single computer, each using a separate terminal.
- MP/M was a fairly advanced operating system for the time, at least on microcomputers. It included a priority-scheduled multitasking kernel (before such a name was used, the kernel was referred to as the nucleus) with memory protection, concurrent input/output (XIOS) and support for spooling and queuing. It also allowed for each user to run multiple programs, and switch between them.

Brief History of CP/M

- The last 8-bit version of CP/M was version 3, often called CP/M Plus, released in 1983. Its BDOS was designed by David K. Brown. It incorporated the bank switching memory management of MP/M in a single-user, single-task operating system compatible with CP/M 2.2 applications. CP/M 3 could therefore use more than 64 KB of memory on an 8080 or Z80 processor. The system could be configured to support date stamping of files. The operating system distribution software also included a relocating assembler and linker.
- CP/M eventually became the de facto standard and the dominant operating system for microcomputers, in combination with the S-100 bus computers. This computer platform was widely used in business through the late 1970s and into the mid-1980s.

Brief History of CP/M

- The operating system was described as a "software bus", allowing multiple programs to interact with different hardware in a standardized way.
- Programs written for CP/M were typically portable among different machines, usually requiring only the specification of the escape sequences for control of the screen and printer.
- This portability made CP/M popular, and much more software was written for CP/M than for operating systems that ran on only one brand of hardware.
- One restriction on portability was that certain programs used the extended instruction set of the Z80 processor and would not operate on an 8080 or 8085 processor.

Brief History of CP/M

- Another was graphics routines, especially in games and graphics programs, which were generally machine-specific as they used direct hardware access for speed, bypassing the OS and BIOS (this was also a common problem in early DOS machines).
- CP/M increased the market size for both hardware and software by greatly reducing the amount of programming required to port an application to a new manufacturer's computer.
- An important driver of software innovation was the advent of (comparatively) low-cost microcomputers running CP/M, as independent programmers and hackers bought them and shared their creations in user groups.

Brief History of CP/M

- Many different companies produced CP/M-based computers for many different markets; InfoWorld stated that "CP/M is well on its way to establishing itself as the small-computer operating system".
- Even companies with proprietary operating systems, such as Heath/Zenith (HDOS), offered CP/M as an alternative for their 8080/Z80-based systems

What is CP/M

What Is CP/M

- CP/M is a monitor control program for microcomputer system development that uses floppy disks or Winchester hard disks for backup storage. Using a computer system based on the Intel 8080 microcomputer, CP/M provides an environment for program construction, storage, and editing, along with assembly and program checkout facilities.
- CP/M can be easily altered to execute with any computer configuration that uses a Zilog Z80 or an Intel 8080 Central Processing Unit (CPU) and has at least 20K bytes of main memory with up to 16 disk drives. Although the standard Digital Research version operates on a single-density Intel MDS 800, several different hardware manufacturers support their own input-output (I/O) drivers for CP/M.
- The CP/M monitor provides rapid access to programs through a comprehensive file management package. The file subsystem supports a named file structure, allowing dynamic allocation of file space as well as sequential and random file access. Using this file system, a large number of programs can be stored in both source and machine executable form.

What Is CP/M

- CP/M is logically divided into several distinct parts:
  - BIOS (Basic I/O System), hardware-dependent
  - BDOS (Basic Disk Operating System)
  - CCP (Console Command Processor)
  - TPA (Transient Program Area)

What Is CP/M - BIOS

- The BIOS provides the primitive operations necessary to access the disk drives and to interface standard peripherals: teletype, CRT, paper tape reader/punch, and user-defined peripherals.
- You can tailor peripherals for any particular hardware environment by patching this portion of CP/M. If a computer manufacturer wanted to be able to run CP/M on their hardware, they would have to create a version of the BIOS that understood the hardware.
- This was the beginning of the Hardware Abstraction Layer that would allow CM/P to run on so many different hardware platforms.

What Is CP/M - BDOS

- The BDOS contains all primitive functions (available to CP/M and user programs) to deal with disk drives, keyboard and screen, etc.
- It provides disk management by controlling one or more disk drives containing independent file directories. The BDOS implements disk allocation strategies that provide fully dynamic file construction while minimizing head movement across the disk during access.
-

What Is CP/M - BDOS

- The BDOS has entry points that include the following primitive operations, which the program accesses:
  - SEARCH looks for a particular disk file by name.
  - OPEN opens a file for further operations.
  - CLOSE closes a file after processing.
  - RENAME changes the name of a particular file.
  - READ reads a record from a particular file.
  - WRITE writes a record to a particular file.
  - SELECT selects a particular disk drive for further operations.
-

What Is CP/M - CCP

- The CCP provides a symbolic interface between your console and the remainder of the CP/M system.
- The CCP reads the console device and processes commands, which include listing the file directory, printing the contents of files, and controlling the operation of transient programs, such as assemblers, editors, and debuggers.

What Is CP/M - TPA

- The last segment of CP/M is the area called the Transient Program Area (TPA). The TPA holds programs that are loaded from the disk under command of the CCP.
- During program editing, for example, the TPA holds the CP/M text editor machine code and data areas. Similarly, programs created under CP/M can be checked out by loading and executing these programs in the TPA.

What Is CP/M - TPA

- Any or all of the CP/M component subsystems can be overlaid by an executing program. That is, once a user's program is loaded into the TPA, the CCP, BDOS, and BIOS areas can be used as the program's data area.
- A bootstrap loader is programmatically accessible whenever the BIOS portion is not overlaid; thus, the user program need only branch to the bootstrap loader at the end of execution and the complete CP/M monitor is reloaded from disk.

CP/M Commands

- There are two types of commands in CP/M
  - Built-In Commands
  - Transient Commands
- If the CCP does not recognize the command as one of the built-in commands, it then searches the current disk for a file of the same name as the command and tries to load and execute it.
- In this way CP/M can be extended by any user developed program as a command.

Built-In Commands

- The built in commands are part of the CCP and are always available from the command prompt
  - **ERA** - Erased one or more files from the disk
  - **DIR** - Displays a directory of filenames on the disk
  - **REN** - Renames a specified file
  - **SAVE** - Saves blocks of memory contents to a file
  - **TYPE** - Displays the contents of a file on the console
  - **USER** - Changes the user number for disk access

Transient Commands

- The transient commands are loaded into the TPA, by the CCP, from disk and executed. Here are some of the typical transient commands bundled with CP/M
  - **STAT** – Lists the number of bytes of storage remaining on the currently logged disk, provides statistical information about particular files, and displays or alters device assignment.
  - **ASM** – Loads the CP/M assembler and assembles the specified program from disk, generating an Intel HEX machine code format.
  - **LOAD** – Loads the file in Intel HEX machine code format and produces a file in machine executable form which can be loaded into the TPA. This loaded program becomes a new command under the CCP.
  - **DDT** – Loads the CP/M debugger into TPA and starts execution.

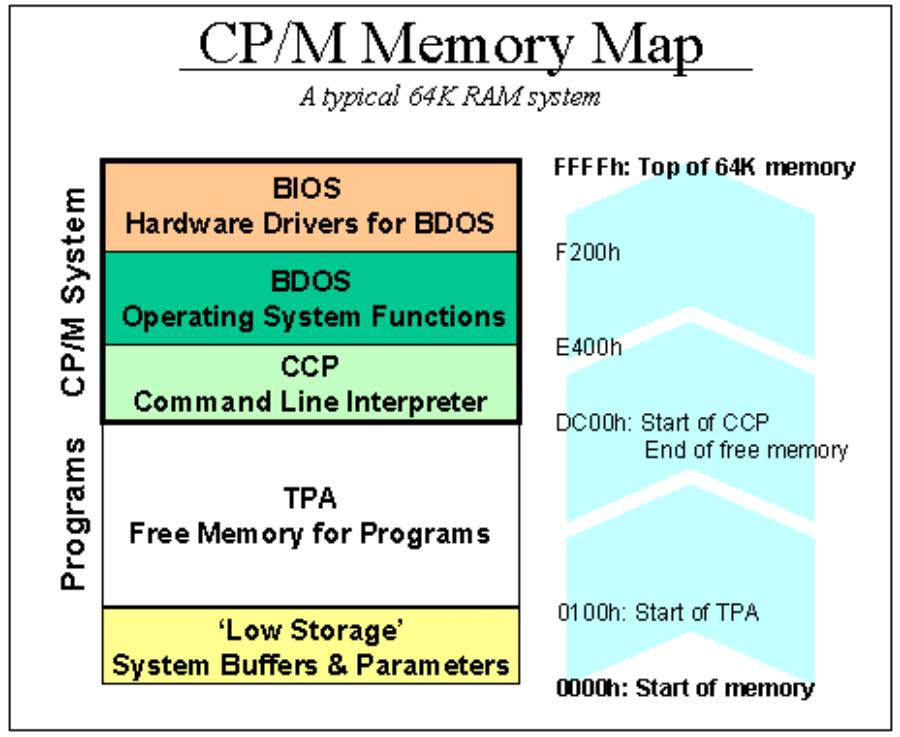
Transient Commands

- **PIP** - Loads the Peripheral Interchange Program for subsequent disk file and peripheral transfer operations.
- **ED** - Loads and executes the CP/M text editor program.
- **SYSGEN** - Creates a new CP/M system disk.
- **SUBMIT** - Submits a file of commands for batch processing.
- **DUMP** - Dumps the contents of a file in hex.
- **MOVCPM** - Regenerates the CP/M system for a particular memory size
-

CP/M Hardware

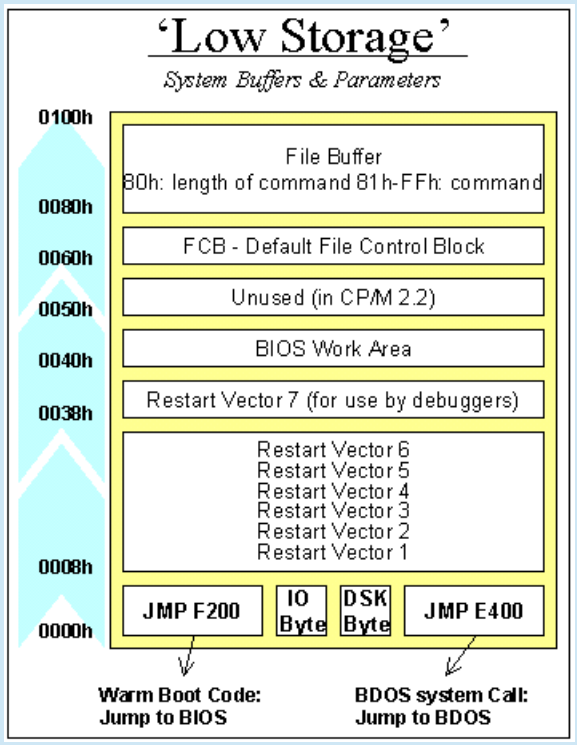
- A minimal 8-bit CP/M system would contain the following components:
  - A computer terminal using the ASCII character set
  - An Intel 8080 (and later the 8085) or Zilog Z80 microprocessor
  - At least 16 kilobytes of RAM, beginning at address 0. The CPU could support up to 64 kilobytes.
  - A means to bootstrap the first sector of the diskette. Either a boot ROM or a boot loader manually entered into memory.
  - At least one floppy-disk drive

CP/M Internals



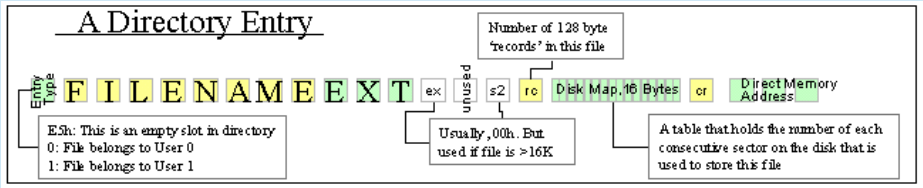
- The first 256 bytes (100 hexadecimal) are used as a scratchpad by the operating system. It contains buffers, parameters and other transient data. This area is called Low Storage.
- Next is free memory up to the bottom of the CCP is the TPA where all programs are executed.
- Then the CCP. This can be overwritten by the application and is reloaded on program exit.
- The BDOS sits between the CCP and the BIOS.
- The BIOS all the way up to the top of memory

CP/M Internals



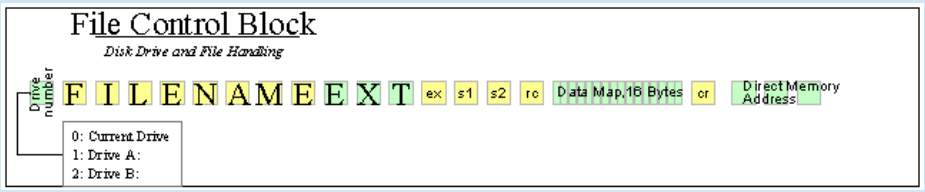
- Low Storage has many CP/M scratch variables
- The Warm Boot vector
- The IO Byte
- Four 2 bit fields that map the 4 logical devices, CON:, LST: RDR:, PUN: to physical devices
- The Current Disk Byte
  High 4 bits current user, Low 4 bits current disk
- The BDOS system call vector
- Reset Vectors
- BIOS Work area
- Default File Control Blocks
  The CCP tries to parse the first two arguments as filenames into FDB1 and FCB2
- DMA Buffer / Command Tail

CP/M Internals



- On floppy disks the format adds 64 directory entries that contain the user number, file name and file allocation information.
- The first byte in a directory entry simply states whether the entry is used or not (if it holds E5h, it's empty). This is actually how the ERA (file delete) command works. Otherwise it is the User number of the file.
- The following bytes hold the filename and extension of the file. ASCII uses only the first 7 bits of each byte. The eighth, upper bit of the three bytes that hold the extension are used to signal whether a file is read-only, invisible, or backed-up (archived).

CP/M Internals



- To manipulate files through the BDOS you need a File Control Block, which mirrors the directory entry structure.
- For opening a file so its data can be read, the FCB needs to be initialized as follows:
  - Enter the drive number in byte 60h, the first byte. 0 = current default drive, 1 = A:, 2 = B:.
  - Enter the filename in the following 11 bytes. 8 Bytes of file name and 3 bytes of file extension.

Programming for CP/M

- All of the services of CP/M are accessed through the BDOS.
- CP/M facilities that are available for access by transient programs fall into two general categories: simple device I/O and disk file I/O.
- Access to the BDOS functions is accomplished by passing a function number and information address through the primary point at location BOOT+0005H.

Programming for CP/M

- For instance, if a program wants to print the character 'A' to the screen:
- ```
  MVI    A,65
  MVI    C,02H
  CALL    0005h
  ```
- First it loads the value 65 (for A) in the accumulator,
- Then loads the value 02h (for BDOS service #, print character) in the C register
- And finally does a CALL 0005 to execute the BDOS request
- CP/M will put the character on the screen, and then return execution to the program.
  ```

Programming Languages with examples

Programming Languages

- There are many programming languages for CP/M.
- The first PL/M, an acronym for Programming Language for Microcomputers, is a high-level language conceived and developed by Gary Kildall in 1973 for Hank Smith at Intel for the Intel 8008. It was later expanded for the newer Intel 8080.
- The 8080 had enough power to run the PL/M compiler, but lacked a suitable form of mass storage. In an effort to port the language from the PDP-10 to the 8080, Kildall used PL/M to write a disk operating system that allowed a floppy disk to be used. This was the basis of CP/M.

Programming Languages

- Assembler was the first available programming language for the processor. But people were hand assembling their code and just toggling in the op–codes to get their programs to run. The assembler and linker / loader were supplied with CP/M
- BASIC gained popularity, particularly for general-purpose programming and educational purposes.
- Pascal, known for its structured programming approach, also gained traction on CP/M, providing an alternative to BASIC and assembly for more complex applications.

Programming Languages

- While other languages like FORTRAN and COBOL were prevalent during the same period, they were more common on larger mainframe systems and not as widely used on CP/M.
- There are even implementations of Forth and Lisp for CP/M.
- Assembly language remained crucial for close-to-hardware tasks, while BASIC and Pascal offered more user-friendly programming environments for general applications.

Programming Languages

- It would be a large undertaking to discuss all of the languages for CP/M development. The ones I am going to cover here are:
  - Assembler
  - BASIC
  - Pascal
  - C
- I will give a brief history, follow up with some personal anecdotes and the conclude each language with a simple Hello World program

Programming Languages

- Here are some of the languages that have been archived and made available for native development:
  - PL/M
  - MS Fortran-80
  - MS COBOL-80
  - Turbo Modula
  - AP/L 2
  - Forth-80
  - Lisp 80
  - SLR
- I have used a few of these languages in different environments and can discuss them if there is any interest.

Assembler

- In computer programming, assembly language (alternatively assembler language or symbolic machine code), often referred to simply as assembly and commonly abbreviated as ASM or asm, is any low-level programming language with a very strong correspondence between the instructions in the language and the architecture's machine code instructions. Assembly language usually has one statement per machine instruction (1:1), but constants, comments, assembler directives, symbolic labels of, e.g., memory locations, registers, and macros are generally also supported.
- Because assembly depends on the machine code instructions, each assembly language is specific to a particular computer architecture.

Assembler

- Sometimes there is more than one assembler for the same architecture, and sometimes an assembler is specific to an operating system or to particular family of operating systems. Most assembly languages do not provide specific syntax for operating system calls, and most assembly languages can be used universally with any operating system, as the language provides access to all the real capabilities of the processor, upon which all system call mechanisms ultimately rest.
- In contrast to assembly languages, most high-level programming languages are generally portable across multiple architectures but require interpreting or compiling, a much more complicated tasks than assembling.

The Assembler Hello World program is just a simple call to the BDOS to output a string.

```
C>TYPE HELLO.ASM
        ORG     100H

BDOS    EQU     0005H                   ; LOCATION OF BDOS ENTRY POINT
BOOT    EQU     0000H                   ; LOCATION OF BOOT REQUEST

START:
        MVI     C,9                     ; BDOS REQUEST 9 - PRINT STRING
        LXI     D,MESSAGE               ; OUR STRING TO PRINT
        CALL    BDOS
        JMP     BOOT                    ; EXIT TO CP/M

MESSAGE:
        DB      13,10,'Hello World from CP/M!',13,10,'$'
        END     START

C>ASM HELLO
CP/M ASSEMBLER - VER 2.0
0126
000H USE FACTOR
END OF ASSEMBLY

C>LOAD HELLO
FIRST ADDRESS 0100
LAST  ADDRESS 0125
BYTES READ    0026
RECORDS WRITTEN 01

C>HELLO

Hello World from CP/M!

C>
```

BASIC

- Beginners All-purpose Symbolic Instruction Code
- BASIC is a family of general-purpose, high-level programming languages designed for ease of use. The original version was created by John G. Kemeny and Thomas E. Kurtz at Dartmouth College in 1963.
- The first microcomputer version of BASIC was co-written by Bill Gates, Paul Allen and Monte Davidoff for their newly formed company, Micro-Soft. This was released by MITS in punch tape format for the Altair 8800 shortly after the machine itself,

The sample program for BASIC is a looped take on the classic Hello World program using CP/M MBASIC

```
F>MBASIC
BASIC-80 Rev. 5.21
[CP/M Version]
Copyright 1977-1981 (C) by Microsoft
Created: 28-Jul-81
34872 Bytes free

Ok

LOAD "HELLO.BAS"

Ok

LIST
10 FOR I = 1 TO 10
20 PRINT "Hello World! From BASIC line";I
30 NEXT I
40 END

Ok

RUN
Hello World! From BASIC line 1
Hello World! From BASIC line 2
Hello World! From BASIC line 3
Hello World! From BASIC line 4
Hello World! From BASIC line 5
Hello World! From BASIC line 6
Hello World! From BASIC line 7
Hello World! From BASIC line 8
Hello World! From BASIC line 9
Hello World! From BASIC line 10

Ok
```

Now, let us use BASCOM to compile the BASIC file into an executable COM file.

```
F>TYPE HELLO.BAS
10 FOR I = 1 TO 10
20 PRINT "Hello World! From BASIC line";I
30 NEXT I
40 END

F>BASCOM =HELLO /E

00000 Fatal Error(s)
24196 Bytes Free

F>L80 HELLO,HELLO/N/E

Link-80  3.44  09-Dec-81  Copyright (c) 1981 Microsoft

Data    4000    4210    <  528>
37990 Bytes Free
[4015   4210       66]

F>HELLO
Hello World! From BASIC line 1
Hello World! From BASIC line 2
Hello World! From BASIC line 3
Hello World! From BASIC line 4
Hello World! From BASIC line 5
Hello World! From BASIC line 6
Hello World! From BASIC line 7
Hello World! From BASIC line 8
Hello World! From BASIC line 9
Hello World! From BASIC line 10

F>
```

Pascal

- Pascal is an imperative and procedural programming language, designed by Niklaus Wirth as a small, efficient language intended to encourage good programming practices using structured programming and data structuring. It is named after French mathematician, philosopher and physicist Blaise Pascal.
- Pascal became very successful in the 1970s, notably on the burgeoning minicomputer market. Compilers were also available for many microcomputers as the field emerged in the late 1970s. It was widely used as a teaching language in university-level programming courses in the 1980s, and also used in production settings for writing commercial software during the same period.

Pascal

- UCSD Pascal formed the basis of many systems, including Apple Pascal. Borland Pascal was not based on the UCSD codebase, but arrived during the popular period of UCSD and matched many of its features. This started the line that ended with Delphi Pascal and the compatible Open Source compiler FPC/Lazarus.
- Turbo Pascal became hugely popular, thanks to an aggressive pricing strategy, having one of the first full-screen IDEs, and very fast turnaround time (just seconds to compile, link, and run). It was written and highly optimized entirely in assembly language, making it smaller and faster than much of the competition.

The Pascal Hello World program is also a looped take on the classic Hello World program using Turbo Pascal

```
G>TYPE HELLO.PAS
program hello;
var no : integer;
begin
 ClrScr;
 for no := 1 to 10 do
   writeln('Hello World from Turbo Pascal Line ', no);
end.

G>TURBO

G>HELLO
Hello World from Turbo Pascal Line 1
Hello World from Turbo Pascal Line 2
Hello World from Turbo Pascal Line 3
Hello World from Turbo Pascal Line 4
Hello World from Turbo Pascal Line 5
Hello World from Turbo Pascal Line 6
Hello World from Turbo Pascal Line 7
Hello World from Turbo Pascal Line 8
Hello World from Turbo Pascal Line 9
Hello World from Turbo Pascal Line 10

G>
```

C

- C is a general-purpose programming language. It was created in the 1970s by Dennis Ritchie and remains very widely used and influential. By design, C's features cleanly reflect the capabilities of the targeted CPUs. It has found lasting use in operating systems code (especially in kernels), device drivers, and protocol stacks, but its use in application software has been decreasing. C is commonly used on computer architectures that range from the largest supercomputers to the smallest micro-controllers and embedded systems.
- C is an imperative procedural language, supporting structured programming, lexical variable scope, and recursion, with a static type system. It was designed to be compiled to provide low-level access to memory and language constructs that map efficiently to machine instructions, all with minimal runtime support. Despite its low-level capabilities, the language was designed to encourage cross-platform programming. A standards-compliant C program written with portability in mind can be compiled for a wide variety of computer platforms and operating systems with few changes to its source code.

C

- It has a static type system. In C, all executable code is contained within subroutines (also called "functions", though not in the sense of functional programming).
- Function parameters are passed by value, although arrays are passed as pointers, i.e. the address of the first item in the array.
- Pass-by-reference is simulated in C by explicitly passing pointers to the thing being referenced.
- C program source text is free-form code. Semicolons terminate statements, while curly braces are used to group statements into blocks.

The C code uses again a looped version of the classic Hello World program.

```
E> type hello.c
Main()
{
    int i;
    for( i = 1; i <= 10; i++ ) {
        printf("Hello World, from Aztec C line %d\n", i);
    }
}

E> cz hello
C Vers. 1.06D Z80 (C) 1982 1983 1984 by Manx Software Systems

E>as hello
8080 Assembler Vers. 1.06D

E>ln hello.o -lc
C Linker Vers. 1.06D
Base: 0100  Code: 1ff0 Data: 01f8 Udata: 0362 Total: 00254e

E> hello
Hello World, from Aztec C line 1
Hello World, from Aztec C line 2
Hello World, from Aztec C line 3
Hello World, from Aztec C line 4
Hello World, from Aztec C line 5
Hello World, from Aztec C line 6
Hello World, from Aztec C line 7
Hello World, from Aztec C line 8
Hello World, from Aztec C line 9
Hello World, from Aztec C line 10

E>
```

Modern CP/M Tools

Modern CP/M Tools

- There are many modern ways to experience CP/M
  - **Modern Hardware –** There are still systems being produced that can run CP/M today.
  - **Emulators –** Software emulation of the processor and hardware allows you to run CP/M in your favorite modern environments
  - **Utilities –** There are tool that run on Windows, Linux, and MacOS that allow you to manage disk images that are compatible with the emulators and with the proper equipment, existing modern and legacy hardware

Modern Hardware

- There are several “new” Z-80 machines out there.
- Small Computer Central has three varieties:
  - Modular backplane and cards. The backplanes come in a few sizes:
    - RCBus 40-pin (standard RC2014 bus)
    - RCBus 80-pin (extended RC2014 bus)
    - Z50Bus (LiNC 50-pin)
  - Expandable single board computers (motherboards)
  - Non-expandable single board computers (SBC)
- A good ROM bios for these systems is RomWBW. This is customizable and can even have a bootable CP/M system disk in the ROM image. The project also has a large selection of CP/M software packaged up as disk images.

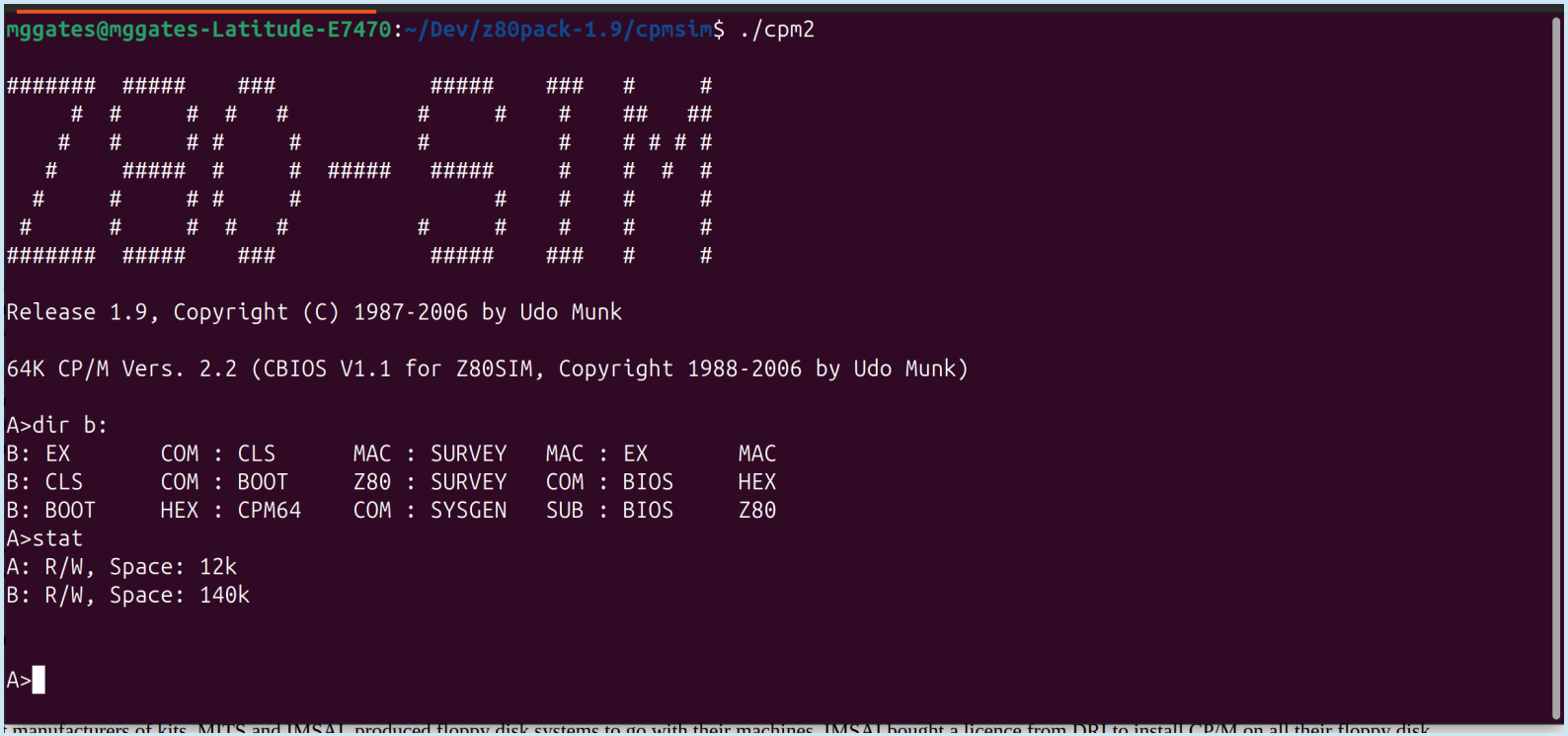
Modern Hardware

- I am using the SC-131 the Z-180 Pocket Computer today to run all of the programming examples. They also have systems with the Z-80 processor.
- CPUVille,Designing, Building, and Selling Obsolete Computers -- for Educational Purposes -- since 2004 also has a Z-80 computer kit and accessories.
- There are even GitHub repositories with new Z-180 systems being designed, such at YAZ180.

Emulators

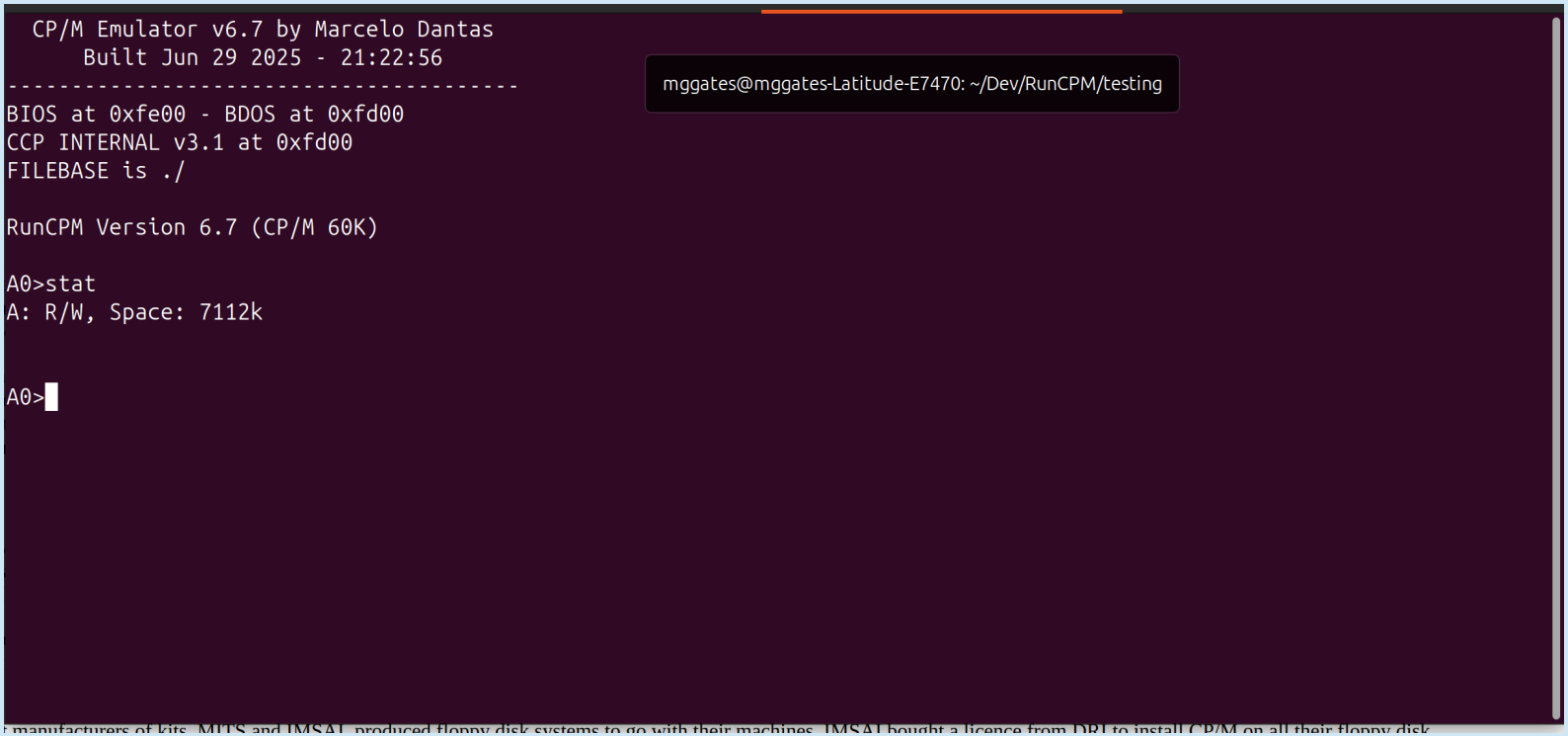
- There are several good software emulators that allow you to run CP/M.
- These are the ones I worked with for this presentation.
- They all can be run under Linux, Windows, and MacOS. With the exception of CPMemu which only supports Linux.
  - Z80pack
  - RunCPM
  - Open simh
  - CPMEmu

Emulator - Z80pack

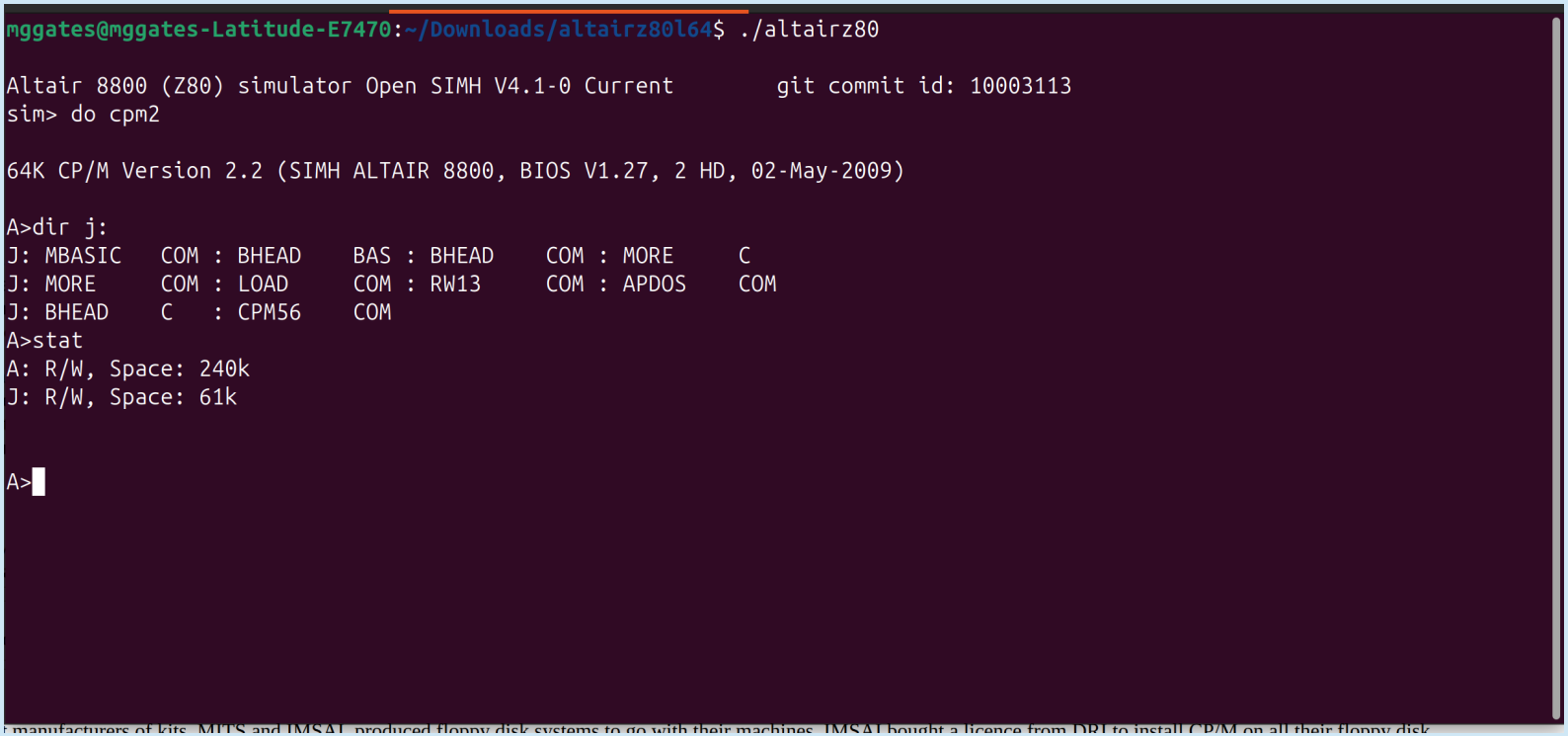


manufacturers of kits, MITS and IMSAI, produced floppy disk systems to go with their machines. IMSAI bought a licence from DRI to install CP/M on all their floppy disk

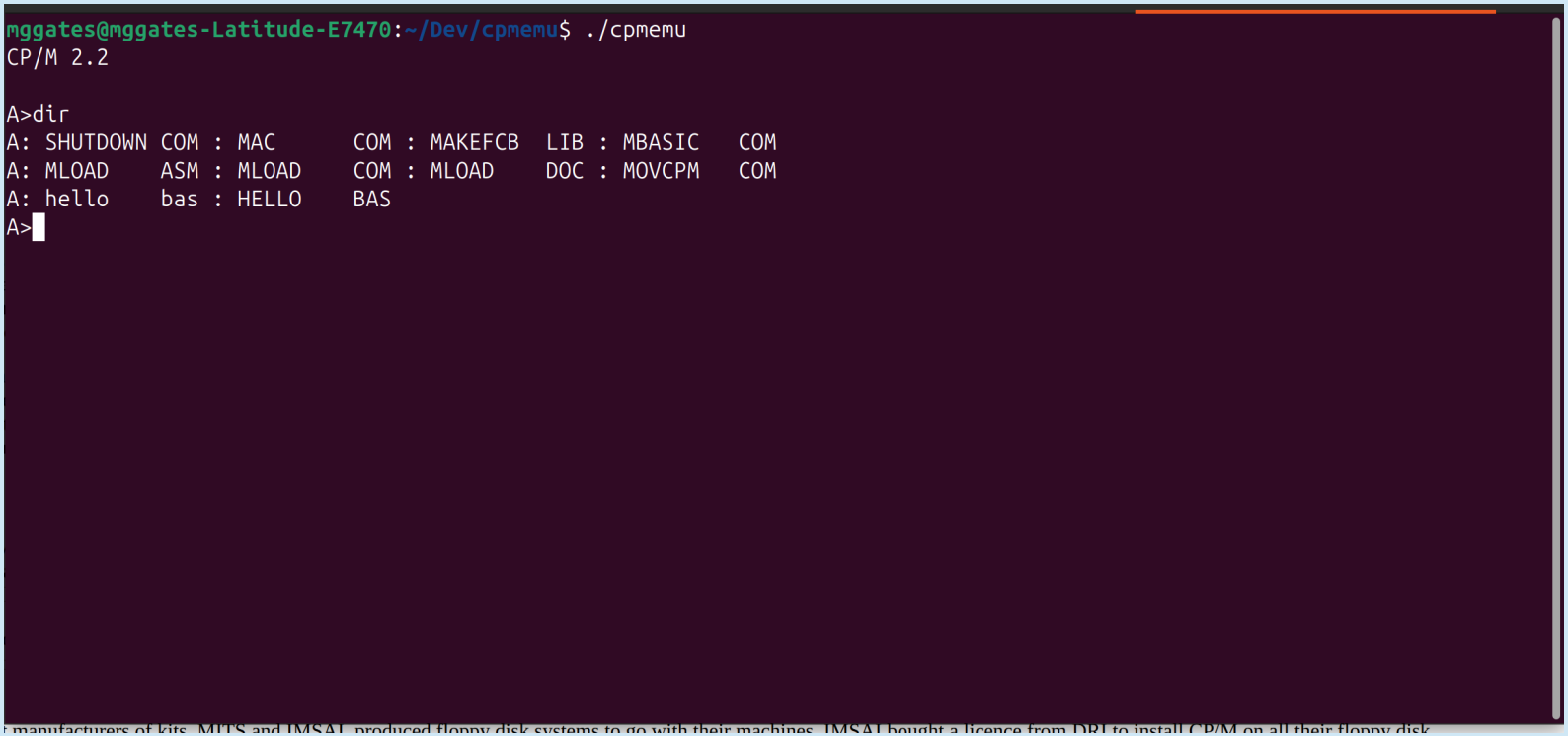
Emulator - RunCPM



Emulator – Open simh



Emulator - CPMemu



manufacturers of kits, MITS and IMSAI, produced floppy disk systems to go with their machines. IMSAI bought a licence from DRI to install CP/M on all their floppy disk

Utilities

- Modern Utilities allow you to
  - Access CP/M disk image files
  - Cross Compile CP/M code on native environments

Utilities - cpmtools

- This package allows you to access CP/M file systems. It can be used for file exchange with a Z80-PC simulator, but it works on floppy devices as well.
- All CP/M file system features are supported. Password protection is ignored, because passwords are easy to decrypt, but a pseudo file [passwd] contains them, if you are curious what your old password has been. The disk label is read as special file [label]. User numbers are specified as user:file.
- Cpmtools should compile and work out of the box on any POSIX compliant system. The source is available as a on GitHub.
- The package also includes manual pages for the commands.

Utilities - cpmtools

- Cpmtools commands available:
  - cpmls - list sorted directory with output similar to ls, DIR, P2DOS DIR and CP/M3 DIR
  - cpmcp - copy files from and to CP/M file systems
  - cpmrm - erase files from CP/M file systems
  - cpmchmod - change file permissions
  - cpmchattr - change file attributes
  - mkfs.cpm - make a CP/M file system
  - fsck.cpm - check and repair a CP/M file system
  - fsed.cpm - view CP/M file system
-

Utilities - ZXCC

- ZXCC is a two-purpose CP/M 2/3 emulator allowing:
  - Hi-Tech C for CP/M to be used as a cross-compiler under Unix.
  - The CP/M build tools (MAC, RMAC, GENCOM, LINK) to be used under DOS or Unix.
- A lot of the functionality of ZXCC is wrapped up in two separate libraries:
  - CPMIO (terminal emulation)
  - CPMREDIR (filesystem emulation).

Utilities - ZXCC

- The libraries are LGPLed and should be fairly easy to use separately from ZXCC. For example, the stable version of CPMREDIR is also used in the JOYCE PCW emulator.
- The main feature of CPMIO is multiple terminal emulations, as seen in MYZ80 under DOS. Current emulations are VT52, VT100 and native (no emulation).
- CPMREDIR allows directories on the host system to appear as CP/M drives. ZXCC uses this for all drives; it is also possible to use it to supply only some CP/M drives, as JOYCE does.

Summary

- CP/M was a leap forward for its time.
- As one of the first Commercial Disk Operating Systems, CP/M found its way on to a large number of systems.
- Some of the top business software started on CP/M.
- There were many programming languages developed for CP/M.
- The research for this talk brought back so many memories.

Questions and Answers

- Feel free to ask any questions about CP/M, Software Development and the tools I have talked about today.
- Additional Resources on GitHub